

IBM BOB 2.0 EXTENSION CHALLENGE

BobMigrate Lite

Automated Legacy JavaScript Modernization
via a Custom MCP Server + Agent Skill

Python MCP Server

9-Step Agent Skill

Zero LLM Token Analysis

OWASP Security Audit

Self-Healing Loop

THE PROBLEM

Legacy JavaScript Is Everywhere

Millions of Node.js codebases still use patterns that are harder to read, test, and maintain — and there's no safe, automated way to fix them.

var declarations

Function-scoped `var` causes hoisting bugs and unclear intent. Modern JS requires `const` / `let`.

LOW SEVERITY

Callback patterns

Error-first callbacks (`function(err, result)`) make async code difficult to reason about, test, or compose.

HIGH SEVERITY

Raw `http.get()` calls

Node's `http.get()` is not Promise-based, blocking adoption of `async/await` and modern `fetch` APIs.

MEDIUM SEVERITY

Manual refactoring is tedious, error-prone, and provides **no guarantee** the output actually works.

THE SOLUTION

One Command. Full Pipeline.

BobMigrate Lite is an IBM Bob 2.0 extension that combines a **custom MCP server** with a **9-step agent skill** to automate the entire migration workflow — from analysis to a verified, audited output.

```
// Trigger the full pipeline in Bob Agent Mode
Modernize sample-app/user_service.js
```

Bob handles the rest: analyze → refactor → test → heal → audit → report.

Python MCP Server

mcp-server/server.py — two deterministic tools registered via .bob/mcp.json. Zero LLM tokens for analysis.

Agent Skill

.bob/skills/modernize/SKILL.md — a 9-step workflow loaded into Bob's context via use_skill.

Custom Command

.bob/commands/modernize.md — slash-command shortcut to trigger the full pipeline.

ARCHITECTURE

How It All Connects

```

| IBM Bob 2.0 Agent |
| |
| .bob/skills/modernize/SKILL.md |
| ↳ 9-step workflow (use_skill) |
| | |
| | calls MCP tools |
| ▼ |
| .bob/mcp.json → mcp-server/server.py |
| | |
| └─ detect_legacy_patterns |
| └─ security_audit |
| |
| Bob Shell: node sample-app/test.js |

```

FastMCP over stdio

Server runs as a subprocess. Bob auto-discovers tools from the JSON schema. No network, no auth, fully local.

Skill as workflow engine

SKILL.md is loaded once per session via `use_skill` and drives all 9 steps — including the self-healing retry loop.

Shell as truth oracle

`execute_command` runs Node.js tests. Exit code 0 = pass. No LLM opinion — real runtime execution.

9-STEP WORKFLOW

The Full Migration Pipeline

- 1 Identify target file** — default: `sample-app/user_service.js`
- 2 detect_legacy_patterns** — MCP tool scans for `var`, `callbacks`, `http.get`. Returns structured JSON findings.
- 3 Snapshot original** — stores `ORIGINAL_CODE` for the before/after summary.
- 4 Refactor** — LLM rewrites: `var`→`const/let`, `callbacks`→`async/await`, `http.get`→`Promise`.
- 5 Verify test compatibility** — confirms `test.js` expects a `Promise` return.
- 6 Execute tests** — runs `node sample-app/test.js` via Bob Shell.
- 7 Self-healing loop** — if test fails, patch and retry up to **3×** before escalating.
- 8 security_audit** — MCP tool checks for `eval()` and hardcoded secrets (OWASP baseline).
- 9 Migration report** — structured summary: issues found, transformations applied, test , audit .

LIVE DEMO RESULTS

Before & After — user_service.js

BEFORE — legacy user_service.js (9 issues detected)

```
// 5× var, 3× callback, 1× http.get var http =  
require('http'); function getUserData(userId, callback) {  
var options = { hostname: '...' }; http.get(options,  
function(err, res) { if (err) return callback(err, null);  
// ... callback(null, user); }); }
```

❌ NODE TEST.JS → EXIT 1

9

Legacy issues detected
in user_service.js

AFTER — modernized (0 issues)

```
// const/let, async/await, Promise const http =  
require('http'); async function getUserData(userId) {  
return new Promise((resolve, reject) => { const options =  
{ hostname: '...' }; http.get(options, (res) => { // ...  
resolve(user); }).on('error', reject); }); }
```

✅ NODE TEST.JS → EXIT 0

✅ OWASP PASSED

0

LLM tokens used
for static analysis

3×

Max self-healing
retry attempts

KEY DIFFERENTIATORS & QUICK START

Why BobMigrate Lite Stands Out

Deterministic validation

Pass/fail is decided by *actual code execution* (`node test.js`), not an LLM opinion. Exit code is the truth.

Zero-token static analysis

Both MCP tools run entirely in Python with regex. Only the structured JSON findings enter LLM context.

Self-healing robustness

The skill retries up to 3× on test failure — reading error output and patching the file each iteration.

OWASP security gate

Every migration ends with a security audit. Hardcoded secrets and `eval()` are caught before the file is finalized.

Get started in 3 steps:

- 1 `git clone <repo> && cd bob-migrate`
Requires Python 3.8+, Node.js 16+
- 2 `bash setup.sh`
Installs deps, smoke-tests MCP server, runs fixture tests
- 3 Open folder in **IBM Bob 2.0** and type:
`Modernize sample-app/user_service.js`

Bob Extension Stack Used

MCP Server · Custom Skill · Slash Command · Bob Shell
(`execute_command`)